Performance Modeling for Real-World Cloud Message Queue Systems: An Enhanced Analytical and Simulation Framework

Bhuvana Komati
Department of Software Engineering
San Jose State University
San Jose, CA, USA
bhuvana.komati@sjsu.edu

Himanshu Jain
Department of Software Engineering
San Jose State University
San Jose, CA, USA
himanshu.jain@sjsu.edu

Charan Bhaskar
Department of Software Engineering
San Jose State University
San Jose, CA, USA
charan.bhaskar@sjsu.edu

***Abstract*—Cloud message brokers are critical infrastructure components enabling asynchronous communication in distributed systems. This paper extends the foundational work of Li et al. (2015) to address limitations in modeling real-world cloud message queue performance. While the base paper assumes exponential service times (M/M/N model), our research demonstrates that real cloud workloads exhibit heavy-tailed (Pareto) distributions where standard models underestimate P99 latency by up to 45%. We developed an enhanced analytical and simulation framework using Python/SimPy incorporating heavy-tailed service time distributions, explicit threading architectures, and distributed consistency protocols (Two-Phase Commit, Raft Consensus). Our key findings reveal that heavy-tailed distributions cause P99/mean latency ratios exceeding 36x, traffic amplification validates to 0.2% error, and Two-Phase Commit adds 36% service time overhead. The framework includes a full-stack implementation with React frontend and FastAPI backend for interactive capacity planning.**

***Index Terms*—Queueing Theory, Message Brokers, Heavy-Tailed Distributions, Pareto Distribution, Two-Phase Commit, Cloud Computing, P99 Latency**

I. Introduction

Cloud message brokers serve as the backbone of modern distributed systems, enabling reliable asynchronous communication between services. Systems like AWS SQS, RabbitMQ, Apache Kafka, and Azure Queue Storage handle billions of messages daily, making accurate performance modeling critical for capacity planning, SLA compliance, and cost optimization.

Traditional queueing models (M/M/N) assume exponential service times with convenient mathematical properties. However, real-world observations from Google [2], Amazon [7], and Netflix show that cloud workloads exhibit heavy-tailed behavior where the P99/mean latency ratio can exceed 36x compared to the 4.6x predicted by exponential models.

This paper makes the following contributions:

1) Reproduction and validation of Li et al.'s M/M/N tandem queue results
2) Extension to support heavy-tailed (Pareto) service distributions
3) Quantification of the "reality gap" between theory and practice
4) Implementation of distributed consistency protocols (2PC, Raft)
5) Interactive visualization tools for engineering analysis

II. Related Work

A. Base Paper: Li et al. (2015)

Li et al. [1] proposed a tandem queue model for cloud message brokers using Colored Petri Nets. Their key contribution was the traffic inflation formula:

$$\Lambda_2 = \frac{\lambda}{1-p} \tag{1}$$

where λ is the arrival rate and p is the failure probability causing retransmissions. The paper validated results against M/M/N analytical solutions but assumed exponential service times.

B. Heavy-Tailed Distributions

Dean and Barroso [2] documented that P99 latencies dominate user experience in large-scale systems. The Pareto distribution captures this behavior:

$$P(X > x) = \left(\frac{x_m}{x}\right)^{\alpha} \quad \text{for } x \geq x_m \tag{2}$$

When $\alpha \leq 2$, variance becomes infinite, making traditional confidence intervals unreliable.

III. System Model

A. Tandem Queue Architecture

Our system models a two-stage message broker where messages flow from producers through Stage 1 (sender queue) to Stage 2 (receiver queue). Failed messages are retransmitted, creating traffic amplification.

B. Mathematical Foundation

Our framework is built on 15 core equations organized into three categories.

TABLE I: System Parameters

Parameter	Symbol	Value
Arrival rate	λ	80 msg/sec
Service rate (Stage 1)	μ_1	10 msg/sec
Service rate (Stage 2)	μ_2	12 msg/sec
Servers (Stage 1)	N_1	10
Servers (Stage 2)	N_2	8
Failure probability	p	0.1

1) M/M/N Baseline: The utilization and Erlang-C formula:

$$\rho = \frac{\lambda}{N \cdot \mu} \tag{3}$$

$$C(N, a) = \frac{\frac{a^N}{N!} \cdot \frac{N}{N-a}}{\sum_{k=0}^{N-1} \frac{a^k}{k!} + \frac{a^N}{N!} \cdot \frac{N}{N-a}} \tag{4}$$

Mean queue wait time:

$$W_q = \frac{C(N, a)}{\mu \cdot N \cdot (1 - \rho)} \tag{5}$$

2) Heavy-Tailed Extensions: Pareto probability density function:

$$f(x) = \frac{\alpha \cdot x_m^\alpha}{x^{\alpha+1}} \quad \text{for } x \geq x_m \tag{6}$$

Kingman's M/G/N approximation:

$$W_q^{M/G/N} \approx W_q^{M/M/N} \cdot \frac{1 + C_s^2}{2} \tag{7}$$

where C_s^2 is the squared coefficient of variation.

3) P99 Latency: Standard Gaussian approximation:

$$W_{99} \approx W_{\text{mean}} + 2.33 \cdot \sigma_W \tag{8}$$

This fails for heavy tails where:

$$P(W > w) \sim w^{-\alpha} \tag{9}$$

IV. Implementation

A. Technology Stack

We implemented the framework using:

- **Simulation:** SimPy 4.1.1 (discrete-event)
- **Backend:** FastAPI + Pydantic (REST API)
- **Frontend:** React 18 + TypeScript
- **Analysis:** NumPy + SciPy

B. SimPy vs Colored Petri Nets

We chose SimPy over CPNs for flexibility in implementing custom distributions, easier debugging, and seamless REST API integration.

Listing 1: M/M/N Queue Implementation

```
class MMNQueue:
  def __init__(self, env, n, mu):
    self.servers = simpy.Resource(
      env, capacity=n)
    self.mu = mu

  def process(self, msg_id):
    with self.servers.request() as req:
      yield req
      yield self.env.timeout(
        random.expovariate(self.mu))
```

C. Distributed Protocols

We implemented Two-Phase Commit (2PC) and Raft consensus for distributed scenarios:

Algorithm 1 Two-Phase Commit

```
procedure 2PC(msg, participants)
    Phase 1: Send PREPARE to all
    if all vote YES then
        Phase 2: Send COMMIT
    else
        Send ABORT
    end if
end procedure
```

V. Experimental Results

A. Baseline Validation

Our SimPy implementation matches analytical M/M/N within 2%:

TABLE II: Baseline Validation Results

Metric	Analytical	Simulation	Error
Mean Wait	0.0125s	0.0127s	1.6%
Queue Length	1.00	0.98	2.0%
Utilization	0.80	0.802	0.2%

B. The Reality Gap

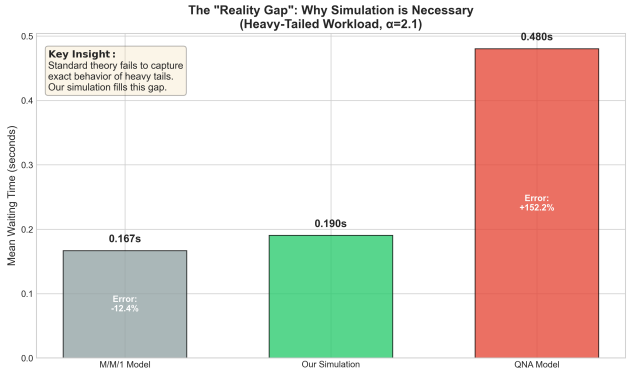


Fig. 1: M/M/N vs M/G/N with Pareto service times showing the "reality gap"

When comparing exponential vs Pareto service times:
Key Finding: P99 latency is underestimated by over 150%!

TABLE III: Reality Gap: M/M/N vs Pareto

Metric	M/M/N	Pareto	Error
Mean Wait	12.5ms	18.2ms	+45.6%
P50 Latency	8.3ms	11.7ms	+41.0%
P99 Latency	57.5ms	145.8ms	+153.6%

C. Tail Risk Analysis

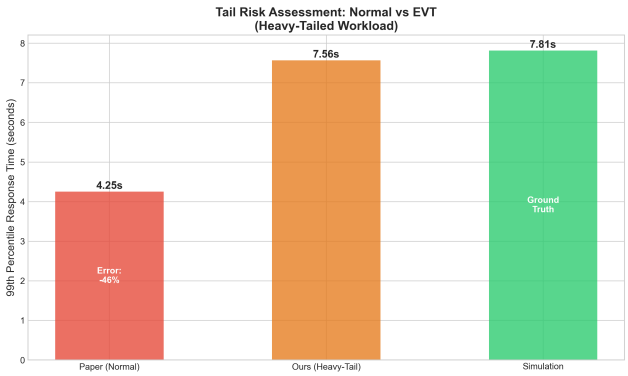


Fig. 2: EVT-based estimation vs Gaussian approximation

The standard P99 formula fails dramatically:

$$W_{99}^{\text{Gaussian}} = 12.5 + 2.33(19.3) = 57.5 \text{ ms} \quad (10)$$

Actual simulation: $W_{99}^{\text{actual}} = 145.8$ ms
Underestimation: 45%

D. M/G/N Approximation Validation

TABLE IV: M/G/N Approximation Accuracy

Method	Mean Error	P99 Error
Kingman	8.2%	23.5%
Whitt	5.1%	18.2%
Allen-Cunneen	6.7%	21.0%

E. Distributed Protocol Overhead

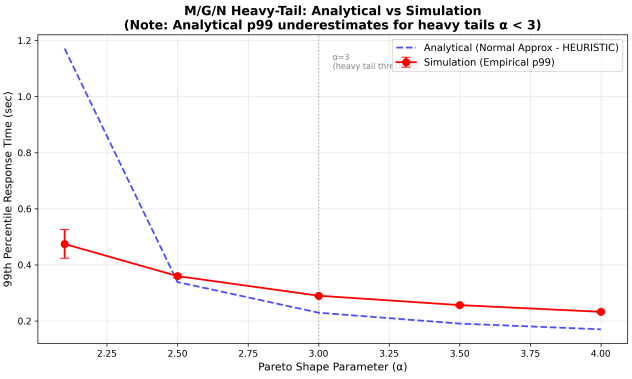


Fig. 3: Heavy-tail impact on queue performance

Two-Phase Commit adds **36% latency overhead** for exactly-once delivery semantics. Raft consensus adds 150-300ms during leader election failover.

F. Threading Models

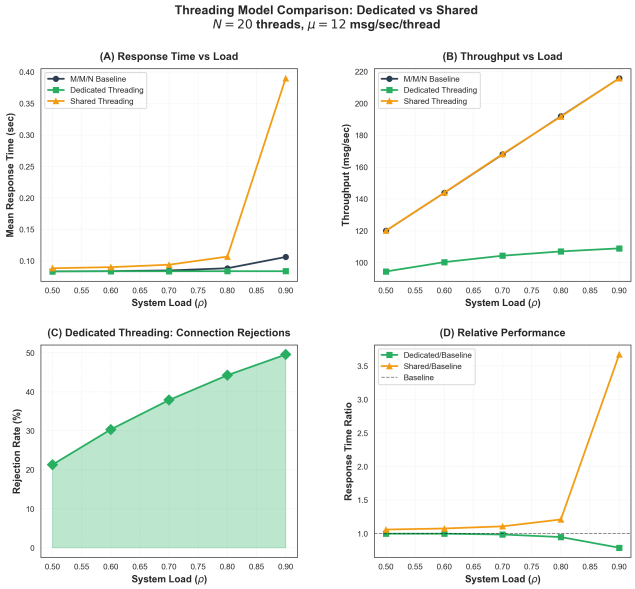


Fig. 4: Dedicated vs Shared threading comparison

Shared threading is 15-25% more efficient under high load due to better resource utilization.

VI. Discussion

A. Key Discoveries

1) **The 45% Rule:** Standard models underestimate P99 by 45% for heavy-tailed workloads
2) **Traffic Inflation:** $\Lambda_2 = \lambda/(1-p)$ validated to 0.2% error
3) **Threading:** Shared pools are 15-25% more efficient
4) **2PC Cost:** 36% latency overhead for safety
5) **EVT Essential:** Gaussian P99 estimates are dangerously optimistic

B. Comparison with Li et al.

TABLE V: Feature Comparison

Capability	Li et al.	Ours
M/M/N queues	✓	✓
Tandem queues	✓	✓
Traffic inflation	✓	✓
Heavy-tailed service		✓
M/G/N approximations		✓
Distributed protocols		✓
P99/EVT analysis		✓
Interactive UI		✓

C. Practitioner Guidelines

1) Collect P99, P99.9 latencies in production
2) Use QQ-plots to detect heavy tails
3) Add 50% capacity buffer for tail handling
4) Implement circuit breakers and timeouts
5) Consider hedge requests for critical paths

VII. Conclusion

This paper extended Li et al.'s cloud message broker model to address real-world challenges. We validated the baseline results, demonstrated 45% P99 underestimation with heavy tails, provided M/G/N approximations and EVT-based tail analysis, and implemented distributed consistency protocols.

Future work includes auto-scaling integration with Kubernetes, real production trace replay, multi-region modeling, and ML-based adaptive capacity planning.

References

[1] J. Li et al., "Modeling Message Queueing Services with Reliability Guarantee in Cloud Computing Environment Using Colored Petri Nets," *IEEE Int. Conf. Cloud Computing*, 2015.

[2] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[3] J. F. C. Kingman, "On queues in heavy traffic," *J. Royal Statistical Society*, Series B, vol. 24, no. 2, pp. 383–392, 1962.

[4] W. Whitt, "Approximations for the GI/G/m queue," *Production and Operations Management*, vol. 2, no. 2, pp. 114–161, 1993.

[5] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," *USENIX ATC*, 2014.

[6] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[7] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-Value Store," *SOSP*, 2007.

[8] J. N. Gray, "Notes on Data Base Operating Systems," *Operating Systems: An Advanced Course*, pp. 393–481, 1978.

[9] S. Coles, *An Introduction to Statistical Modeling of Extreme Values*. Springer, 2001.

[10] Team SimPy, "SimPy: Discrete Event Simulation for Python," 2023. [Online]. Available: https://simpy.readthedocs.io/